

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería · Ingeniería de Software

AVANCE 2 · ARQUITECTURA DE LA SOLUCIÓN

Plataforma de comercio electrónico para la venta de ropa juvenil personalizable

Caso: Coral Shop S.A.C.

Datos generales	
Curso	Desarrollo Web Integrado
Docente	Joel Ronald Vilca Chambi
Grupo	Grupo 1
Integrantes	Choquehuanca Marrufo, Liam Lennon Damián Valdivia, Diego Aarón Loayza Gerónimo, Juan Franco Villanueva Montalvo, Apolo Chris Campos Sulca, Jian Pier
Año	2026

Lima - Perú

Índice

Índice	2
1. Introducción	3
1.1 Objetivo del avance	3
1.2 Estado del proyecto	3
1.3 Tecnologías	3
2. Patrones de arquitectura aplicados	5
2.1 Arquitectura en capas	5
2.1.1 El restaurante en capas	5
2.1.2 Caso práctico: crear un pedido	6
2.1.3 Respuestas de error uniformes	8
2.2 Arquitectura cliente-servidor	8
2.3 Modelo-Vista-Controlador	9
2.4 Arquitectura dirigida por eventos	10
2.5 Cómo conviven los cuatro patrones	11
3. Implementación del avance	12
3.1 Estructura de paquetes	12
3.2 Endpoints implementados	12
3.3 Reglas de negocio implementadas	12
3.4 Modelo de datos	13
3.5 Pruebas y verificación	15
4. Próximos pasos	16
5. Conclusiones	16
Anexo A. Diagramas entidad-relación	17

1. Introducción

1.1 Objetivo del avance

Este segundo avance presenta la arquitectura de Coral Shop tal como quedó implementada, a la luz de los cuatro patrones de arquitectura revisados en clase: arquitectura en capas, cliente-servidor, modelo-vista-controlador (MVC) y arquitectura dirigida por eventos. Para cada patrón se explica la idea, dónde aparece en el código real (paquetes, clases y endpoints) y qué problema concreto resuelve en la tienda.

El caso que recorre todo el documento es el corazón del negocio: un cliente personaliza 12 polos con su logo bordado en el pecho, los reparte entre tallas S, M y L, crea el pedido, lo paga y el administrador lo avanza hasta ENTREGADO. Es la misma historia con la que se verificó el sistema de punta a punta.

1.2 Estado del proyecto

El desarrollo se organiza en fases. Las tres primeras están terminadas e integradas a la rama principal del repositorio; las siguientes se describen en la sección 4.

Fase	Contenido	Estado
0 · Ordenar la casa	Código muerto eliminado, interfaz en español y soles, backend reorganizado en capas, manejo uniforme de errores	Terminada
1 · Modelo de datos	Migración V5: tipos de producto, personalización, mayoreo, direcciones, pedidos, historial de estados y pagos (25 tablas)	Terminada
2 · API del flujo de compra	Catálogo filtrado, personalizador, diseños, cotización, pedidos, pago simulado, máquina de estados y administración	Terminada
3 · Frontend del flujo de compra	Personalizador, carrito, checkout y «Mis pedidos» en React	Siguiente
4 · Evento de dominio	OrderStatusChanged con notificaciones y auditoría	Diseñada
5 · Calidad y demo	Pruebas, datos de demostración, colección Postman y documentación	Pendiente
6 · Pasarela real	Pago en sandbox con confirmación por webhook	Pendiente

1.3 Tecnologías

Capa	Tecnología	Relación con lo visto en clase
Vista	React 19 + Vite 8 + Tailwind 4 + React Router 7	Ocupa el lugar de la JSP: la vista se ejecuta en el navegador y recibe JSON.
Controlador	Spring Boot 3.5 (Spring MVC) sobre Java 21	Spring MVC se apoya en un único Servlet, el DispatcherServlet, que reparte cada petición al @RestController que corresponde.
Contenedor web	Apache Tomcat embebido	El mismo contenedor de Servlets de la sesión de

Capa	Tecnología	Relación con lo visto en clase
		contenedores web, empaquetado dentro de la aplicación.
Acceso a datos	JdbcTemplate (Spring JDBC) + JPA para la cuenta	JdbcTemplate es JDBC con PreparedStatement y manejo automático de conexiones; nunca se concatena SQL.
Base de datos	PostgreSQL 16 + Flyway (migraciones V1 a V5)	Integridad referencial y restricciones CHECK en la propia base.
Seguridad	Spring Security: sesión (JSESSIONID), BCrypt, CSRF y roles	El session scope y la cookie JSESSIONID de la clase de JSP y MVC.

2. Patrones de arquitectura aplicados

Esta sección recorre, uno por uno, los patrones revisados en clase. Los diagramas usan nombres reales de clases, endpoints y tablas del repositorio.

2.1 Arquitectura en capas

La arquitectura en capas divide el sistema en niveles con una responsabilidad cada uno: presentación, negocio y datos. La regla que la hace funcionar es que cada capa conoce solo a la capa de abajo. En Coral Shop cada módulo funcional (catálogo, pedidos, pagos, diseños...) tiene sus propios paquetes controller, service y repository, de modo que la separación se ve en la estructura de carpetas.

Capa	Responsabilidad	En Coral Shop	Si algo falla
Presentación	Recibir la petición HTTP y validar su FORMATO	@RestController + DTO con @Valid (CreateOrderRequest, AddressRequest...)	400 · datos con formato inválido
Negocio	Aplicar las REGLAS en una transacción	@Service + @Transactional (OrderService, PricingService, QuantityPolicy, OrderStatusPolicy)	422 · regla violada 409 · conflicto (stock)
Datos	Leer y escribir en la base, solo con parámetros	@Repository con JdbcTemplate (OrderRepository, ProductRepository)	409 · restricción de la base

2.1.1 El restaurante en capas

La dinámica de clase compara las capas con un restaurante. El mozo toma el pedido y revisa que esté completo, el cocinero aplica la receta y el almacenero trae los ingredientes de la despensa. El mozo nunca entra a la despensa. En Coral Shop el mozo es el controller, el cocinero el service, el almacenero el repository y la despensa PostgreSQL.

FIGURA A1 - ARQUITECTURA EN CAPAS

El restaurante en capas aplicado a Coral Shop

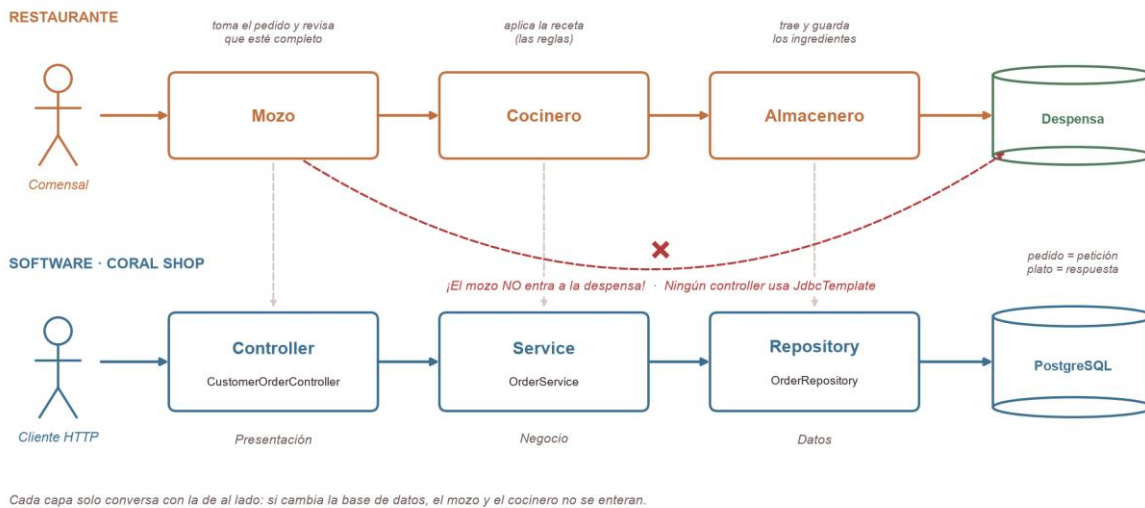


Figura 1. El restaurante en capas aplicado a Coral Shop.

COMPROBACIÓN EN EL CÓDIGO

Ningún controller del backend recibe JdbcTemplate ni escribe SQL: todo el SQL vive en las clases @Repository. Es la versión en código de «el mozo no entra a la despensa». Si mañana cambia la base de datos, solo se tocan los repositorios.

2.1.2 Caso práctico: crear un pedido

Igual que en el ejemplo de la Mesa de Partes Digital, la petición atraviesa las tres capas en orden y cada una responde con un código HTTP distinto cuando algo no cuadra. La Figura 2 muestra el recorrido de POST /api/orders.

FIGURA A2 · ARQUITECTURA EN CAPAS

Crear un pedido en Coral Shop: el recorrido por las tres capas

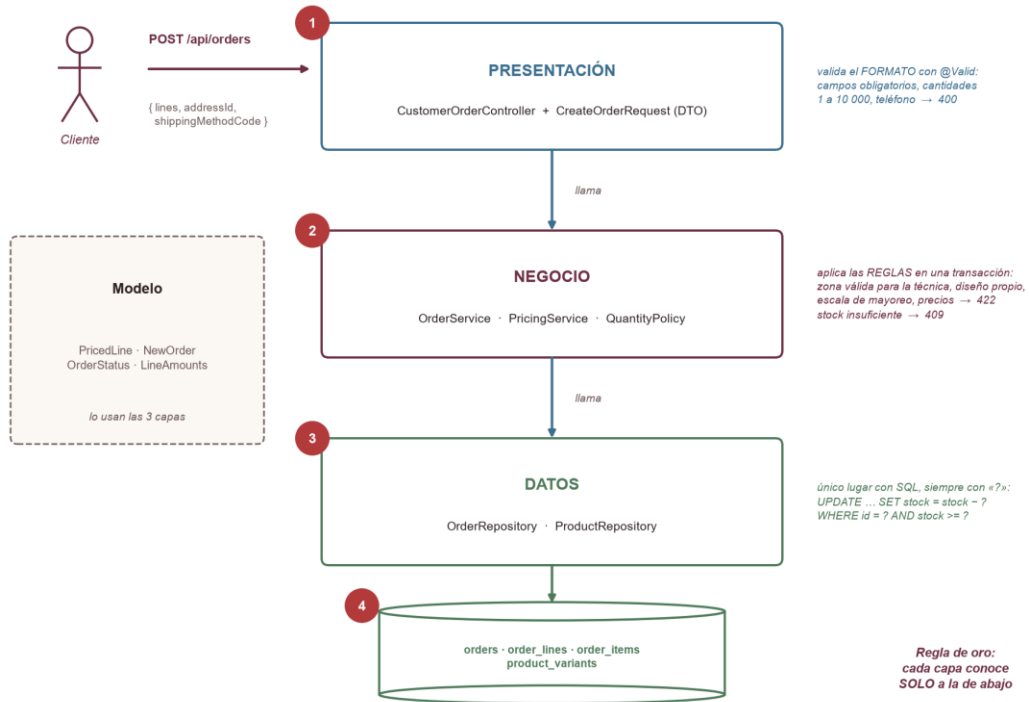


Figura 2. Recorrido de la creación de un pedido por las tres capas.

- **Presentación.** CustomerOrderController recibe el cuerpo y CreateOrderRequest lo valida con @Valid: líneas no vacías, cantidades entre 1 y 10 000, código de envío obligatorio, teléfono con formato. Si algo falla responde 400 sin tocar la base.
- **Negocio.** OrderService abre una transacción. Con PricingService comprueba que el producto esté activo y sea personalizable, que cada zona admita la técnica elegida, que el diseño sea del propio cliente y recalcula todos los precios en el servidor; QuantityPolicy elige la escala de mayorero. Una regla rota responde 422.
- **Datos.** OrderRepository y ProductRepository descuentan el stock con una actualización condicional (UPDATE ... WHERE stock >= ?) y guardan el pedido, sus líneas, zonas, ítems y el primer registro del historial. Si una talla no alcanza, la transacción se deshace completa y se responde 409.

El modelo (PricedLine, NewOrder, OrderStatus, LineAmounts) no pertenece a ninguna capa: son objetos de datos inmutables que las tres comparten, igual que la entidad Trámite en el ejemplo de clase.

2.1.3 Respuestas de error uniformes

Un único `@RestControllerAdvice` traduce las excepciones de todas las capas a un JSON con la misma forma `{ status, message, errors }`. Los mensajes de la tabla son respuestas reales obtenidas al verificar la API.

Código	Capa que lo produce	Ejemplo real
400	Presentación	Cotización sin líneas; archivo que no es imagen; diseño de más de 5 MB
404	Negocio	Un cliente pide el detalle de un pedido ajeno: «Pedido no encontrado»
409	Negocio / Datos	«Stock insuficiente para POLO-...-S (S, Blanco): pediste 21»
422	Negocio	«La zona ESPALDA no admite bordado...»; pasar un pedido de PAGADO a ENTREGADO

2.2 Arquitectura cliente-servidor

En el modelo cliente-servidor varios clientes piden servicios a un servidor que concentra los datos y las reglas. Coral Shop tiene tres tipos de cliente (la tienda, el panel de administración y las herramientas de prueba) y un único servidor Spring Boot, que es el único que habla con PostgreSQL.

FIGURA A3 · ARQUITECTURA CLIENTE-SERVIDOR

Varios clientes, un servidor de aplicación y una base de datos

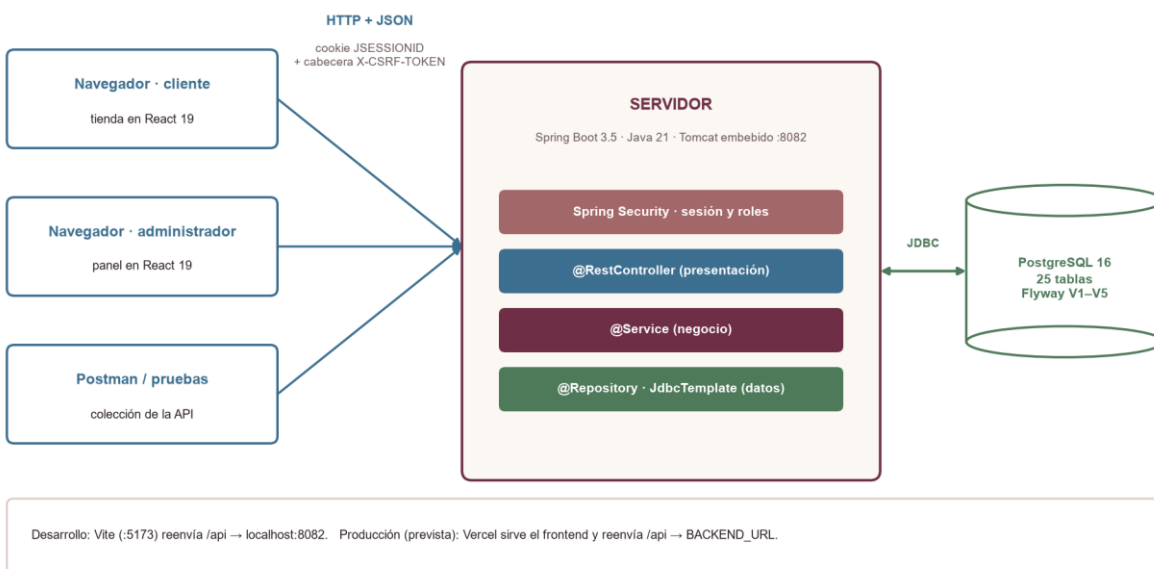


Figura 3. Arquitectura cliente-servidor de Coral Shop.

Aspecto	Decisión	Por qué
Protocolo	HTTP con cuerpos JSON en /api/...	El navegador y Postman usan el mismo contrato.

Aspecto	Decisión	Por qué
Sesión	Cookie JSESSIONID emitida por Spring Security	Se reutiliza el session scope de clase; el rol (cliente o administrador) viaja en la sesión del servidor.
CSRF	Token en la cabecera X-CSRF-TOKEN para todo POST, PUT, PATCH y DELETE	Impide que otra página envíe peticiones con la sesión del usuario.
Confianza	El servidor recalcula precios y stock (D10)	El cliente puede ser manipulado; el servidor nunca acepta importes enviados por el navegador.
Mismo origen	Vite en desarrollo y Vercel en producción reenvían /api al servidor	El navegador solo ve un dominio: la cookie y el CSRF funcionan sin configurar CORS.

2.3 Modelo-Vista-Controlador

MVC separa los datos y sus reglas (modelo), lo que ve el usuario (vista) y quien coordina ambos (controlador). En clase se presentó con JavaBeans, Servlets y JSP. Coral Shop conserva los tres roles; la diferencia es que la vista ya no es una JSP renderizada en el servidor sino una aplicación React que se ejecuta en el navegador.

FIGURA A4 - MODELO-VISTA-CONTROLADOR

MVC con React como vista y Spring como controlador

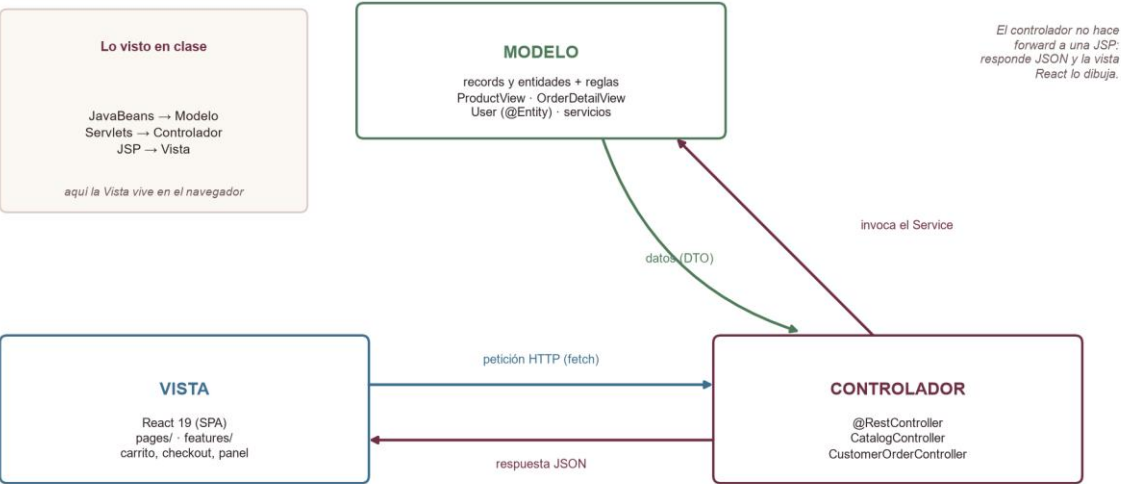


Figura 4. MVC con React como vista y Spring como controlador.

Rol MVC	En clase	En Coral Shop
Modelo	JavaBeans	Records inmutables (ProductView, OrderDetailView, PricedLine), la entidad User de JPA y los servicios que aplican las reglas
Vista	JSP	Pantallas React en pages/ y features/: catálogo, ficha, carrito, checkout y panel

Rol MVC	En clase	En Coral Shop
Controlador	Servlet	@RestController (CatalogController, CustomerOrderController, AdminOrderController...) atendidos por el DispatcherServlet

El controlador ya no hace forward a una JSP: devuelve JSON y la vista React lo dibuja. Gracias a eso la misma API sirve a la tienda, al panel y a las pruebas automáticas.

2.4 Arquitectura dirigida por eventos

En la arquitectura dirigida por eventos un componente publica un hecho («el pedido cambió de estado») en un bus, y otros componentes suscritos reaccionan sin que quien publica los conozca. Así se agregan efectos nuevos sin modificar el código que ya funciona.

ESTADO ACTUAL

Este patrón todavía no está implementado: hoy no existen eventos, suscriptores ni bus. Se diseñó para la fase 4 sobre el mecanismo de eventos que Spring ya incluye, sin agregar un broker externo.

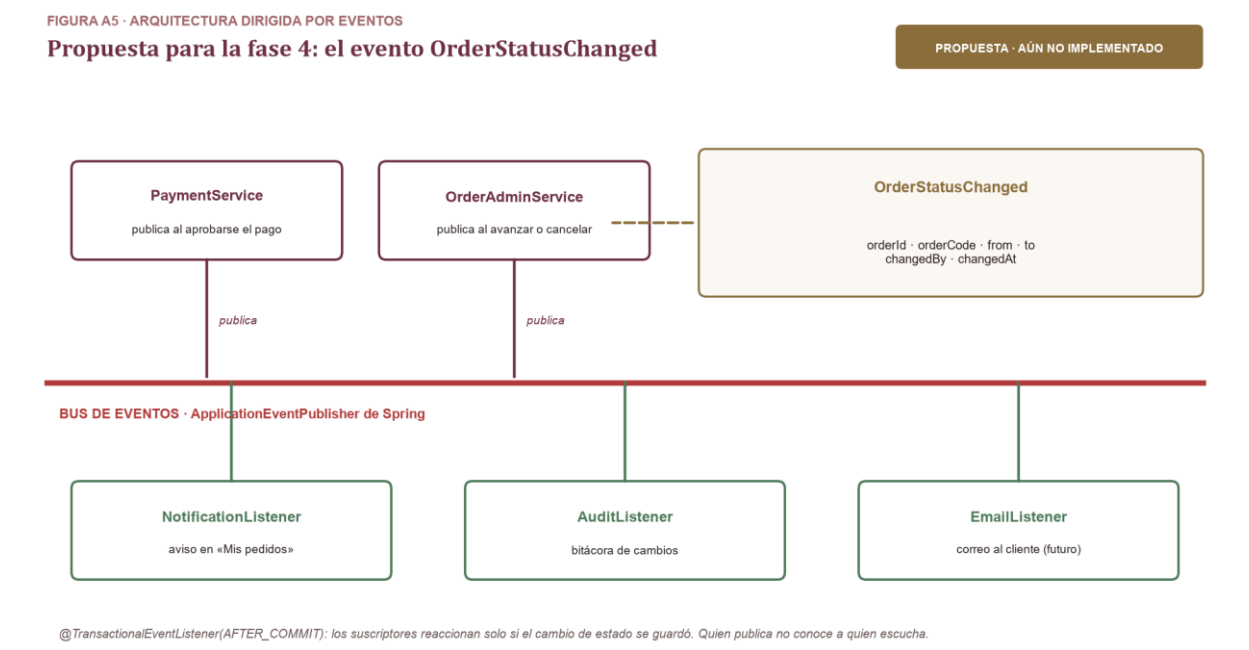


Figura 5. Propuesta del evento OrderStatusChanged para la fase 4.

Elemento	Diseño
Evento	OrderStatusChanged { orderId, orderCode, from, to, changedBy, changedAt }
Publicadores	PaymentService (pago aprobado → PAGADO) y OrderAdminService (avance o cancelación)
Bus	ApplicationEventPublisher de Spring, dentro del mismo proceso

Elemento	Diseño
Suscriptores	NotificationListener (aviso en «Mis pedidos»), AuditListener (bitácora) y, más adelante, EmailListener (correo al cliente)
Garantía	@TransactionalEventListener(AFTER_COMMIT): solo se reacciona si el cambio se guardó

Se descarta por ahora un broker como Kafka o RabbitMQ: con un solo servidor y pocos suscriptores, el bus en memoria de Spring da el mismo desacoplamiento sin infraestructura adicional. Si la tienda creciera a varios servicios, el evento ya definido se podría publicar en un broker sin cambiar a los publicadores.

2.5 Cómo conviven los cuatro patrones

Los patrones no compiten: cada uno describe el sistema desde un nivel distinto y se aplican a la vez.

Patrón	Nivel que describe	Dónde se ve	Beneficio para Coral Shop
Cliente-servidor	Despliegue: qué corre dónde	React en el navegador, Spring Boot en el servidor, PostgreSQL detrás	Reglas y datos protegidos en un solo lugar
MVC	Interacción con el usuario	React (vista), @RestController (controlador), records y servicios (modelo)	La misma API sirve a la tienda, al panel y a las pruebas
Capas	Organización del servidor	Paquetes controller / service / repository de cada módulo	Cambios acotados; errores 400, 422 y 409 bien separados
Eventos	Reacciones a hechos del negocio	OrderStatusChanged (fase 4)	Notificar y auditar sin tocar el flujo de pedidos

3. Implementación del avance

3.1 Estructura de paquetes

El backend se organiza por funcionalidad y, dentro de cada una, por capa. Son 13 módulos y 118 clases; ningún paquete mezcla capas.

```
backend/src/main/java/com/coralshop/
├── auth/                config · controller · dto · service      (registro, sesión, seguridad)
├── catalog/             controller · dto · model · repository · service
├── customization/       controller · dto · model · repository · service
├── design/              controller · dto · model · repository · service
├── pricing/             controller · dto · model · service      (QuantityPolicy, PricingService)
├── order/               controller · dto · model · repository · service
├── payment/             controller · dto · model · repository · service
├── address/             controller · dto · repository · service
├── shipping/            controller · dto · repository · service
├── user/                controller · dto · model · repository · service
├── stats/               controller · dto · repository · service
├── health/              controller · repository · service
├── common/              dto (paginación) · exception (GlobalExceptionHandler)
└── backend/src/main/resources/db/migration/ V1 ... V5 (Flyway)
```

3.2 Endpoints implementados

Método y ruta	Acceso	Para qué sirve
GET /api/products?type=&category=&q=&page=	público	Catálogo filtrado y paginado en el servidor
GET /api/products/{id}/customization-options	público	Técnicas, zonas válidas con recargo y vista previa, escalas de mayoreo
POST /api/quotes	público	Cotiza el carrito con precios recalculados y errores por línea
POST /api/designs · GET /api/designs/{id}/image	cliente	Sube el diseño (PNG/JPG ≤ 5 MB) y lo muestra a su dueño o al administrador
GET/POST/PUT/DELETE /api/addresses	cliente	Libreta de direcciones del Perú
POST /api/orders	cliente	Crea el pedido en una sola transacción
POST /api/orders/{code}/payment	cliente	Pago simulado detrás de la interfaz PaymentGateway
GET /api/orders/me[{code}]	cliente	Mis pedidos y su línea de tiempo
GET /api/orders · PUT /api/orders/{id}/status	admin	Bandeja de pedidos y máquina de estados
GET/PUT/PATCH /api/admin/products/{id}	admin	Edición y activación de productos
POST/PUT/DELETE /api/categories	admin	Categorías; no se elimina una en uso
GET /api/users · PUT /api/users/{id}/role	admin	Cuentas y roles

3.3 Reglas de negocio implementadas

- **Precio en el servidor.** Unitario = precio base + costo de la técnica + recargos de zona; el total de la línea aplica el descuento de la escala. El navegador nunca envía importes.
- **Mayoreo por escalas.** 1–2 unidades sin descuento, 3–5 con 5 %, 6–11 con 10 % y 12 o más con 15 %. Un paquete puede repartirse entre tallas y colores.

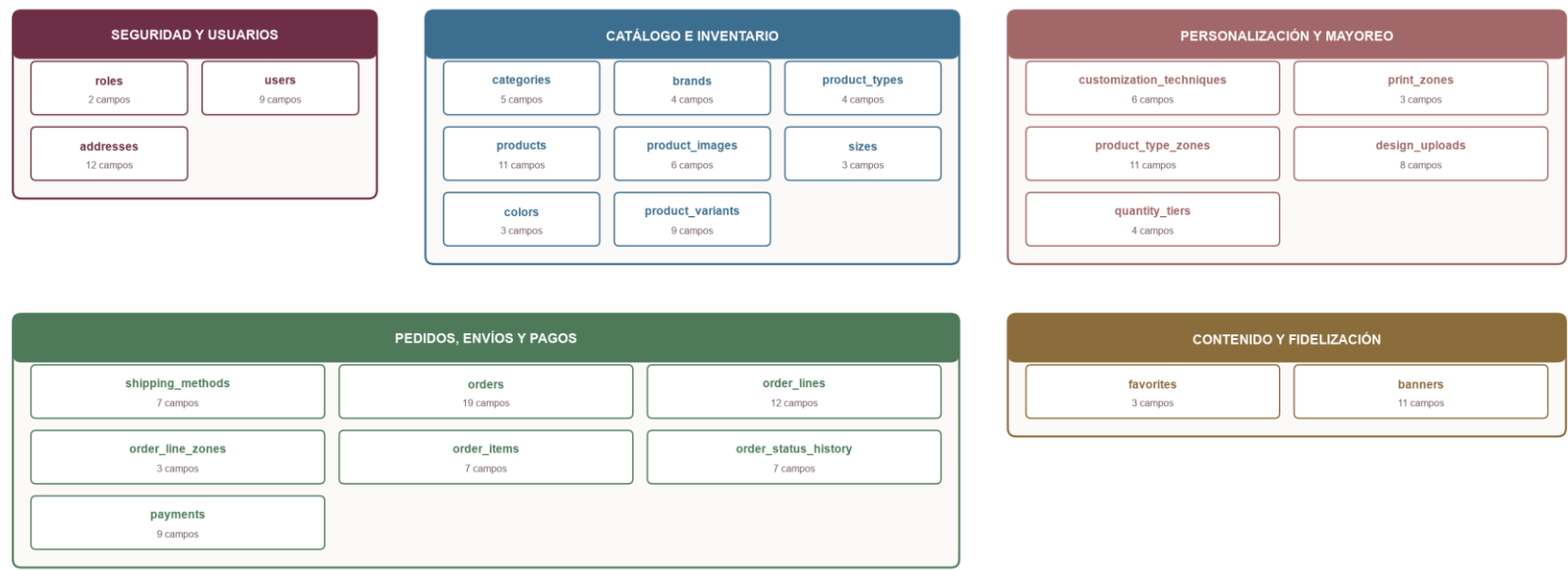
- **Zonas por tipo de producto.** Cada tipo define qué zona admite qué técnica; el bordado tiene menos zonas y tamaños más chicos que el estampado.
- **Stock sin sobreventa.** Se descuenta con una actualización condicional; si dos clientes compran la última unidad a la vez, solo uno lo consigue.
- **Máquina de estados.** PENDIENTE_PAGO → PAGADO → EN_PRODUCCION → LISTO_PARA_ENVIO → ENVIADO → ENTREGADO, o CANCELADO desde los dos primeros; cancelar repone el stock y PAGADO solo se alcanza con un pago aprobado.
- **Pago seguro.** El pedido queda bloqueado durante el cobro, no se puede pagar dos veces y los datos de la tarjeta no se guardan.

3.4 Modelo de datos

La migración V5 completó el modelo: 25 tablas, 178 campos y 29 claves foráneas, agrupadas en cinco módulos. Los diagramas entidad-relación completos están en el Anexo A.

FIGURA 10 · MAPA GLOBAL DEL MODELO DE DATOS

25 tablas agrupadas en 5 módulos funcionales



25 tablas

178 campos

29 claves foráneas

5 módulos funcionales

El detalle de campos, tipos y restricciones de cada tabla se muestra en las figuras 8 y 9 y en el diccionario de datos de la sección 2.4.

Figura 6. Mapa global del modelo de datos.

3.5 Pruebas y verificación

Verificación	Alcance	Resultado
Pruebas unitarias (JUnit 5)	QuantityPolicy, importes de línea, OrderStatusPolicy, detección del tipo de imagen y pasarela simulada	40 de 40 en verde
Recorrido de punta a punta	Cotizar → subir diseño → pedido → pago rechazado y aprobado → admin hasta ENTREGADO, contra PostgreSQL	64 de 64 controles
Reglas rotas	Cada regla violada responde 400, 404, 409 o 422 con un mensaje claro	Incluido en los 64
Frontend	ESLint y build de producción	Sin errores

4. Próximos pasos

- **Fase 3 · Frontend del flujo de compra.** Personalizador con vista previa sobre la foto del producto, carrito por diseño, checkout con direcciones y envío, pago y «Mis pedidos».
- **Fase 4 · Evento de dominio.** Implementar `OrderStatusChanged` con sus suscriptores de notificación y auditoría (sección 2.4).
- **Fase 5 · Calidad y demo.** Datos de demostración ficticios, colección Postman, informe y presentación finales.
- **Despliegue.** Frontend en Vercel configurando la variable `BACKEND_URL`; el backend, en el servicio que se elija.

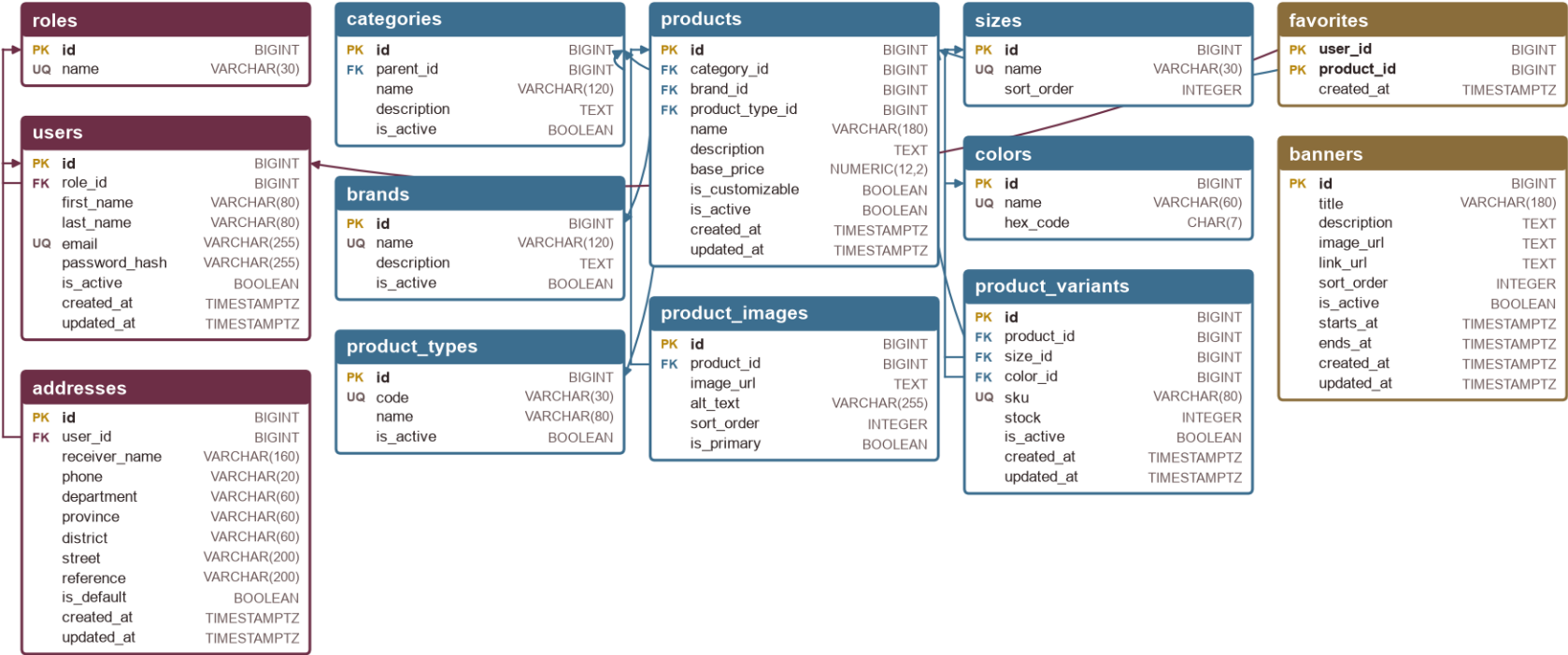
5. Conclusiones

- La arquitectura en capas no quedó solo en el papel: está en la estructura de paquetes y se puede comprobar en el código, porque ningún controller toca la base de datos y cada capa responde con su propio código de error.
- El modelo cliente-servidor, con el servidor como única autoridad sobre precios y stock, es lo que permite vender con seguridad aunque el navegador pueda ser manipulado.
- MVC se mantiene con una adaptación: la vista pasó del servidor (JSP) al navegador (React), y el controlador responde JSON. Eso permite que una misma API atienda a la tienda, al panel y a las pruebas.
- La arquitectura dirigida por eventos está diseñada pero no implementada; se incorporará en la fase 4, cuando hay un efecto concreto que desacoplar: avisar al cliente y auditar cada cambio de estado.

Anexo A. Diagramas entidad-relación

Modelo completo con todos los campos, generado desde el mismo esquema que crean las migraciones.

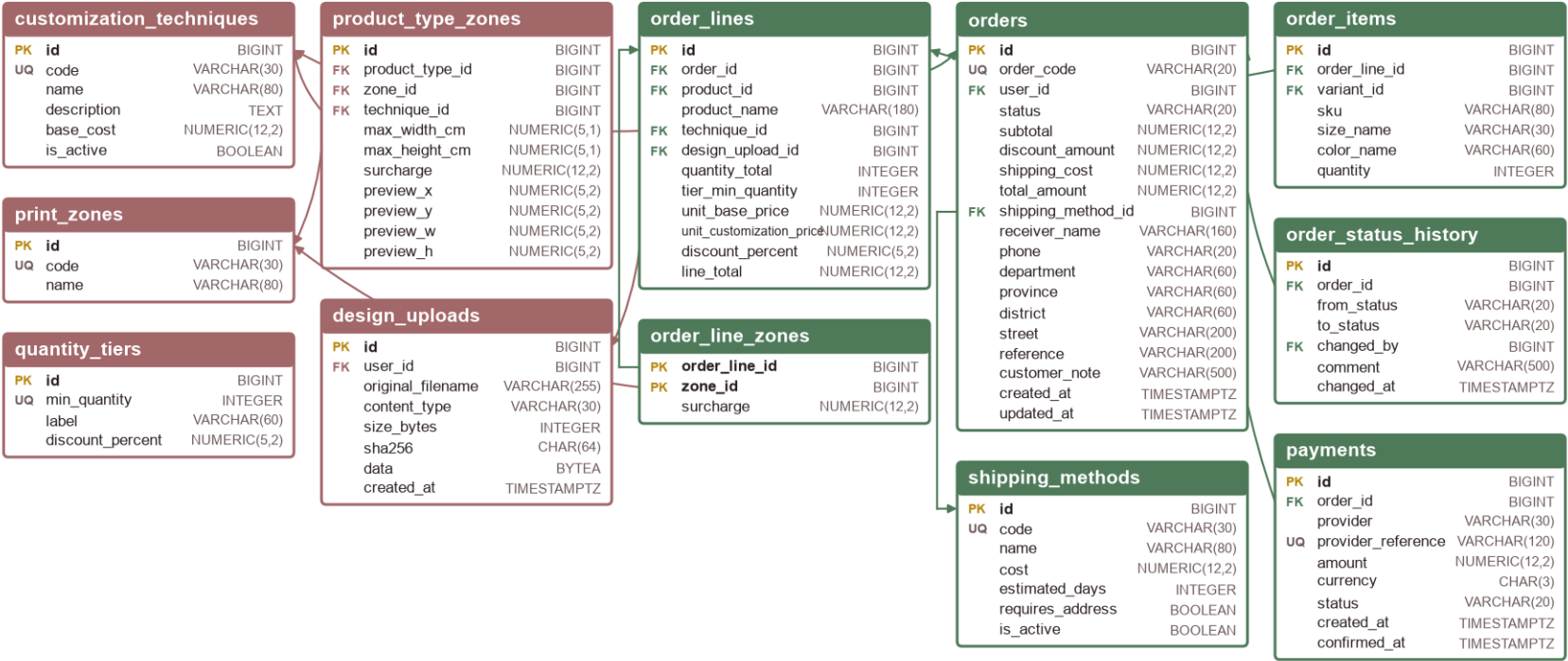
FIGURA 8 · DISEÑO DE BASE DE DATOS
Modelo entidad-relación (1 de 2): usuarios, catálogo e inventario



PK clave primaria **FK** clave foránea (la flecha apunta a la tabla referenciada) **UQ** restricción de unicidad
El stock no vive en products sino en product_variants: cada combinación de producto, talla y color tiene su propio SKU y su propio stock.

Figura A.1. Usuarios, catálogo e inventario.

FIGURA 9 · DISEÑO DE BASE DE DATOS
Modelo entidad-relación (2 de 2): personalización, mayoreo, pedidos y pagos



PK clave primaria FK clave foránea (la flecha apunta a la tabla referenciada) UQ restricción de unicidad
order_lines y order_items copian nombre, SKU, talla, color y precios al comprar: el pedido queda inmutable aunque el catálogo cambie después.

Figura A.2. Personalización, mayoreo, pedidos y pagos.